

Plan de pruebas de software

*Análisis y Diseño de una Plataforma SaaS Multiempresa para la
Gestión del Reclutamiento, Evaluación y Selección de Personal – Caso
de estudio: Colegio Andino de Huancayo*

Fecha: 30/09/2026

Tabla de contenido

Historial de versiones	3
Información del proyecto	3
Aprobaciones.....	3
Resumen ejecutivo.....	3
Alcance de las pruebas	4
Elementos de pruebas	4
Nuevas funcionalidades a probar.....	4
Pruebas de regresión	5
Funcionalidades a no probar.....	5
Enfoque de pruebas (estrategia).....	5
Criterios de aceptación o rechazo.....	6
Criterios de aceptación o rechazo.....	6
Criterios de suspensión.....	6
Criterios de reanudación	6
Entregables	7
Recursos	7
Requerimientos de entornos – Hardware	7
Requerimientos de entornos – Software.....	7
Herramientas de pruebas requeridas.....	7
Personal	8
Entrenamiento	8
Planificación y organización	8
Procedimientos para las pruebas.....	8
Matriz de responsabilidades	9
Cronograma	9
Premisas	9
Dependencias y Riesgos	9
Referencias	10
Glosario	10

Estado del documento: versión 1.0 del 30/09/2026, elaborada por el equipo con asistencia de IA. No está aprobada ni firmada; la aprobación académica está pendiente y no hay validación institucional del Colegio Andino de Huancayo.

Historial de versiones

Fecha	Versión	Autor	Organización	Descripción
30/09/2026	1.0	Equipo del proyecto (redacción asistida por Claude)	Universidad Continental — Pruebas y Calidad de Software, NRC 28607	Primera versión del plan maestro de pruebas (fase F29D), sobre la plantilla del curso, para la versión v1.1 del sistema.

Información del proyecto

Campo	Valor
Empresa / Organización	Universidad Continental — Facultad de Ingeniería, Ingeniería de Sistemas e Informática. Proyecto académico del curso Pruebas y Calidad de Software (NRC 28607)
Proyecto	Análisis y Diseño de una Plataforma SaaS Multiempresa para la Gestión del Reclutamiento, Evaluación y Selección de Personal – Caso de estudio: Colegio Andino de Huancayo
Fecha de preparación	30/09/2026
Cliente	Colegio Andino de Huancayo, como caso de estudio académico. No hay relación contractual ni validación institucional del proceso ni del sistema
Patrocinador principal	No aplica: proyecto académico sin financiamiento
Gerente / Líder de proyecto	Sin asignación formal registrada en el repositorio: responsabilidad compartida del equipo (Coronacion Meza Fredy; Peña Arroyo Anthony; Vila Meza Luis Antonio)
Gerente / Líder de pruebas de software	Sin asignación formal: responsabilidad compartida del equipo. La ejecución y el registro de la F29F los hizo el asistente de IA del equipo bajo su dirección
Docente	Dr. Maglioni Arana Caparachin

Aprobaciones

El plan **no está aprobado ni firmado**. La tabla indica quién debe aprobarlo; no se simula ninguna firma ni aprobación institucional.

Nombre y apellido	Cargo	Departamento u organización	Fecha	Firma
Dr. Maglioni Arana Caparachin	Docente del curso	Universidad Continental	Pendiente	Pendiente — sin firma
Integrantes del equipo	Autores del plan	Universidad Continental (NRC 28607)	Pendiente	Pendiente — sin firma
RR. HH. / Administración del Colegio Andino	Usuario del caso de estudio	Colegio Andino de Huancayo	No aplica	No aplica: no hay validación institucional

Resumen ejecutivo

Este es el **plan maestro** de pruebas de la plataforma SaaS multiempresa de reclutamiento, evaluación y selección de personal, en su versión v1.1 (rama develop, commit bc44303). Cubre la

línea base funcional RF-01 a RF-27, los casos de uso CU-01 a CU-20, los diez RNF académicos del Formato 07 y el componente experimental RF-29.

- **Propósito:** definir qué se prueba, cómo, con qué criterios y con qué evidencia se da por aceptada la versión, y dejar trazado cada requisito hasta sus casos de prueba (F29E) y su ejecución (F29F).
- **Relación con el proyecto:** el plan acompaña la entrega académica final. Las pruebas automatizadas existen desde la Fase 3 (TDD) y el plan las formaliza; no describe un esfuerzo nuevo ni reemplaza la evidencia histórica.
- **Restricciones:** equipo de tres integrantes, un solo equipo de cómputo con Docker, sin entorno de producción, sin datos reales (solo ficticios) y sin validación institucional del Colegio.
- **Límites declarados:** no se mide cobertura de código, no hay pruebas de carga ni de disponibilidad (RNF-06 y RNF-07 no verificados) y la aceptación por el usuario institucional no se realizó.

Alcance de las pruebas

Elementos de pruebas

Áreas funcionales y componentes que se prueban (componentes del Formato 11, CMP-xx):

Área funcional	Componentes	RF
Requerimientos de personal	CMP-04	RF-01 a RF-04
Vacantes y publicación	CMP-05	RF-05 a RF-07
Postulantes y postulaciones	CMP-06, CMP-07	RF-08 a RF-15
Evaluaciones y entrevistas	CMP-08	RF-16 a RF-19
Ranking y comparación (apoyo)	CMP-09	RF-20 a RF-22
Decisión humana, selección y cierre	CMP-10, CMP-11	RF-23 a RF-26
Auditoría	CMP-13	RF-27
Autenticación, roles y multiempresa	CMP-02, CMP-03	Transversal (RNF-01, RNF-02)
Notificaciones y cola	CMP-12, CMP-16	RF-04, RF-11, RF-15, RF-17, RF-26
Interfaz web	CMP-01	Todos (E2E y componentes)
Servicio de riesgo operacional	CMP-17	RF-29 (EXPERIMENTAL / PROPUESTO)

Nuevas funcionalidades a probar

Funcionalidades de la versión v1.1 respecto de la v1.0 académica (tag v1.0.0-academic), vistas desde el usuario:

- **Plazo objetivo de la vacante** (target_completion_at): RR. HH. registra la fecha objetivo del proceso (Fase 16).
- **Tarjeta de riesgo operacional** en la ficha de la vacante (RF-29, EXPERIMENTAL): muestra el riesgo de demora del proceso o un panel descriptivo si el servicio no está disponible. No evalúa personas ni cambia el ranking.
- **Interfaz rediseñada** con movimiento reducido, tablas accesibles en móvil y portada con experiencia CSS 3D (Fases 18 a 21).

Pruebas de regresión

Toda la línea base RF-01 a RF-27 entra en regresión, porque las fases de v1.1 tocaron la interfaz, la ficha de vacante y la configuración del entorno:

- la suite completa de PHPUnit (unitarias y de integración, con autorización por rol y aislamiento entre organizaciones);
- la suite E2E de Cypress, desde el inicio de sesión hasta el flujo completo de reclutamiento;
- la integración continua en GitHub Actions, que en cada push a develop y main ejecuta build, TypeScript y PHPUnit;
- los defectos corregidos DEF-01 a DEF-13, cada uno con su prueba de regresión (docs/defects.md).

Funcionalidades a no probar

Funcionalidad	Razón	Riesgo asumido
RF-28 (candidato)	No está implementado	Ninguno sobre el software; se declara NO IMPLEMENTADO
Rendimiento y carga (RNF-06)	No hay herramienta de carga ni SLA definido	Comportamiento desconocido con volumen real; RNF-06 queda NO VERIFICADO
Disponibilidad y recuperación (RNF-07)	No hay entorno de producción ni respaldos probados	RNF-07 queda NO VERIFICADO
Envío real de correo y SMS	El correo usa el driver log (fuera de alcance)	Entregabilidad real no probada
Despliegue en la nube, SuperAdmin y facturación	Fuera del alcance del proyecto	Ninguno para la entrega académica
Aceptación por el usuario institucional	No hay validación con RR. HH. / Administración del Colegio	El AS-IS sigue preliminar y el TO-BE, propuesto
Pruebas de penetración	Fuera del alcance académico	Solo seguridad básica automatizada
Validez institucional del modelo RF-29	Solo hay datos sintéticos	El modelo sigue EXPERIMENTAL; no se autoriza en producción

Enfoque de pruebas (estrategia)

Estrategia en niveles, con prioridad de la automatización y registro de la evidencia real de cada ejecución. Los datos son siempre ficticios: fábricas de PHPUnit, DemoSeeder para Cypress y el dataset sintético del servicio ML.

Nivel / tipo	Herramienta	Alcance	Automatización
Unitarias	PHPUnit (tests/Unit, 6 archivos)	Estados, validadores de puntajes y ponderaciones, servicio de ranking	Automatizada
Integración / funcionales	PHPUnit (tests/Feature, 42 archivos) sobre PostgreSQL 17 real	RF-01 a RF-27, autorización, notificaciones y auditoría	Automatizada
Seguridad básica	PHPUnit (autenticación, roles, soporte E2E) y Cypress E2E-12	Acceso por rol, límites de intentos, reglas negativas	Automatizada
Multitenencia	PHPUnit (CrossTenantAccessTest, OrganizationScopeTest) y Cypress E2E-11	Ninguna organización accede a datos de otra	Automatizada
UI / E2E	Cypress 15.3.0 (20 specs) en el entorno aislado app-e2e	Flujos por rol y flujo completo	Automatizada
Componentes frontend	Vitest (vp test, 6 archivos)	Componentes de interfaz y accesibilidad	Automatizada
Estático y build	tsc --noEmit y npm run build	Tipos y compilación del frontend	Automatizada

Nivel / tipo	Herramienta	Alcance	Automatización
Regresión continua	GitHub Actions (tests.yml)	Build, TypeScript y PHPUnit en cada push	Automatizada
ML experimental	pytest (18 archivos) y PHPUnit tests/Feature/M1	Dataset, entrenamiento, contrato congelado, API e integración RF-29	Automatizada
Aceptación	Recorrido manual por rol (Fase 8) y flujo E2E-13	Flujo de punta a punta	Parcial: sin aceptación institucional
No funcionales	Cypress E2E-16 a E2E-19 y medición puntual de la F25	Accesibilidad, responsive y tiempos puntuales	Parcial: sin carga ni disponibilidad

Configuraciones: equipo Windows 11 con Docker Desktop, navegador Electron 136 headless, viewport 1280×800 (y móvil en E2E-16). Nivel de regresión: completo en cada ejecución de cierre.

Criterios de aceptación o rechazo

Criterios de aceptación o rechazo

El plan se da por completado y la versión se acepta **solo si se cumplen todos** estos criterios, medidos en la ejecución de la F29F:

ID	Criterio de aceptación
CA-01	PHPUnit: 0 pruebas fallidas. Las únicas omitidas son las 8 de funciones del starter kit desactivadas.
CA-02	Cypress: 20 specs y 0 tests fallidos, sin reintentos.
CA-03	Vitest y pytest: 0 pruebas fallidas.
CA-04	TypeScript sin errores y build del frontend sin errores.
CA-05	Integración continua en verde en develop.
CA-06	Cada RF-01 a RF-27 tiene al menos un caso de prueba automatizado ejecutado y aprobado (F29E y F29F).
CA-07	Ningún defecto de severidad Alta abierto (F29G).

Rechazo: cualquier prueba fallida sin clasificar, una regresión real, un RF sin caso de prueba aprobado o un defecto de severidad Alta abierto. La cobertura de código **no** es criterio: no se mide con herramienta.

Criterios de suspensión

- El entorno Docker no está healthy (servicios app, postgres, redis, app-e2e o queue-e2e).
- La base de datos de pruebas no es la aislada (reclutamiento_testing para PHPUnit, reclutamiento_e2e para Cypress): se detiene la ejecución para no afectar los datos de demostración (DEF-13).
- Un defecto bloquea un flujo principal e impide ejecutar los casos que dependen de él.
- La integración continua falla por una causa externa (servicio de GitHub o dependencia de red).

Criterios de reanudación

- El entorno vuelve a estar healthy y se repite la suite completa afectada, no solo el caso que falló.

- La causa del fallo está clasificada (regresión real, flaky, ambiente, dependencia externa o documental). Si es una regresión real, se corrige con autorización del equipo y con su prueba de regresión.
- Toda ejecución repetida se registra junto a la anterior; no se sustituye ni se oculta el primer resultado.

Entregables

Entregable	Ubicación
Plan de pruebas (este documento)	docs/academico/plan-pruebas/
Casos de prueba y matriz de trazabilidad RF → CU → CP → prueba → evidencia (F29E)	docs/academico/casos-prueba/
Ejecución QA: registros de cada herramienta, resumen y matriz CP → resultado (F29F)	docs/academico/qa-final/
Registro de defectos y métricas de calidad (F29G)	docs/academico/metricas-calidad/
Evidencia TDD (ciclos RED → GREEN)	docs/tdd-evidence.md
Registro histórico de defectos DEF-01 a DEF-13	docs/defects.md
Reportes de herramientas: salida de PHPUnit, Cypress, Vitest, pytest, TypeScript, build y CI	docs/academico/qa-final/evidencias/

Recursos

Requerimientos de entornos – Hardware

Recurso	Especificación (verificada el 30/09/2026)
Equipo de pruebas	AMD Ryzen 9 5900X (12 núcleos), 31,9 GB de RAM, Windows 11 Pro
Máquina virtual de Docker Desktop	24 CPU y 16,7 GB de memoria asignados
Integración continua	Ejecutor ubuntu-latest de GitHub Actions
Red	Acceso a internet para GitHub, imágenes Docker y dependencias; servicios locales publicados solo en 127.0.0.1 salvo app

Requerimientos de entornos – Software

Software	Versión
Docker / Docker Compose	29.7.2 / v5.5.1
Contenedor app	PHP 8.4.25, Node 22.23.2, Laravel 13
Base de datos y caché	PostgreSQL 17.11, Redis 7.4.11
Frontend	React 19, TypeScript, Inertia 3, Tailwind 4, Vite (vp)
Servicio ML	Python 3.12.5 en ml-service/.venv (fuera de Docker Compose)
Cypress	Imagen cypress/included:15.3.0, Electron 136
Bases de prueba	reclutamiento_testing (PHPUnit) y reclutamiento_e2e (Cypress); nunca la de demostración

Herramientas de pruebas requeridas

- **PHPUnit** (vía `php artisan test`): unitarias e integración.
- **Cypress 15.3.0**: E2E en navegador, desde su imagen oficial.
- **Vitest** (vía `vp test`): componentes de React.
- **pytest**: servicio ML experimental.

- **TypeScript** (tsc --noEmit) y **Vite** (npm run build).
- **GitHub Actions**: integración continua (.github/workflows/tests.yml).
- **Git**: control de versiones y trazabilidad de cada ejecución al commit probado.
- Técnica: **TDD** (RED → GREEN → REFACTOR) para cada regla de negocio.

Personal

Rol	Quién	Función en las pruebas
Equipo del proyecto (3)	Coronacion Meza Fredy; Peña Arroyo Anthony; Vila Meza Luis Antonio	Decide el alcance, revisa resultados, aprueba correcciones y es responsable de la entrega
Implementador asistido por IA	Claude (Claude Code)	Escribe pruebas y documentos, ejecuta las suites y registra la evidencia, bajo la dirección del equipo
Auditor asistido por IA	Codex	Revisa cada fase de forma independiente antes de integrarla
Docente	Dr. Maglioni Arana Caparachin	Revisión académica del plan y de sus resultados

Entrenamiento

Necesidades de entrenamiento para operar el plan (no se afirma que se hayan cubierto con cursos formales):

- Pruebas de Laravel con PHPUnit sobre PostgreSQL y fábricas de datos.
- Cypress con entornos aislados y reset de datos.
- Docker Compose: levantar, verificar salud y aislar las bases de prueba.
- pytest y el contrato congelado del experimento ML.
- Lectura de la matriz de trazabilidad y registro de defectos.

Planificación y organización

Procedimientos para las pruebas

- **Desarrollo**: TDD. Cada regla nueva empieza con una prueba que falla (RED observado) y termina en verde; se registra en docs/tdd-evidence.md.
- **Ejecución de cierre**: con el árbol de trabajo limpio, se ejecutan en orden: configuración de Compose, PHPUnit, TypeScript, build, Vitest, pytest y Cypress. Se guarda la salida completa, el comando, la hora y el código de salida.
- **Comandos reales**: docker compose exec app php artisan test · docker compose exec app npx tsc --noEmit · docker compose exec app npm run build · docker compose exec app npx vp test --run · ml-service/.venv/Scripts/python.exe -m pytest · npm run cy:run.
- **Fallos**: se clasifican (regresión real, flaky, ambiente, dependencia externa, documental o no aplicable). Nunca se ocultan, y el runtime no se corrige sin autorización del equipo.
- **Defectos**: se registran como DEF-NN con síntoma, causa, corrección y prueba de regresión.
- **Integración**: rama feature/* desde develop, auditoría y merge --no-ff con la CI en verde.

Matriz de responsabilidades

Matriz RACI (R: responsable, A: aprueba, C: consultado, I: informado).

Actividad	Equipo del proyecto	Claude (implementador)	Codex (auditor)	Docente
Definir alcance y criterios	A	R	C	I
Escribir pruebas automatizadas (TDD)	A	R	C	I
Ejecutar suites y registrar evidencia	A	R	I	I
Clasificar fallos y registrar defectos	A	R	C	I
Corregir regresiones del runtime	A	R (con autorización)	C	I
Auditar la fase	I	C	R	I
Integrar y publicar	A	R	C	I
Revisión académica	R	I	I	A

Cronograma

Hitos de pruebas con sus fechas reales (fecha de integración en develop). Las actividades aún no realizadas no tienen fecha.

Hito	Actividad	Fecha	Depende de
H1	Base multiempresa, roles y RF-01 a RF-27 con TDD (Fases 1 a 7)	13/09/2026	—
H2	Validación manual en navegador (Fase 8)	13/09/2026	H1
H3	Suite E2E con Cypress (Fase 9) y portabilidad Docker (Fase 10)	13/09/2026	H2
H4	Servicio ML: pruebas y validación (Fases 15 a 17)	21/09/2026	H3
H5	QA visual y accesibilidad (Fase 21)	23/09/2026	H4
H6	QA global de release v1.1 (Fase 25)	25/09/2026	H5
H7	Plan de pruebas (F29D) y casos de prueba (F29E)	30/09/2026	H6
H8	Ejecución QA final (F29F), defectos y métricas (F29G)	30/09/2026	H7
H9	Auditoría final de F29D a F29H	Pendiente	H8

Premisas

- Los datos son ficticios y no se usa información personal real.
- El entorno Docker del equipo representa el entorno de ejecución; no existe un entorno de producción.
- Las pruebas automatizadas existentes son válidas como casos de prueba porque su trazabilidad a los RF está verificada.
- La decisión final de selección es humana (RF-23): ninguna prueba espera que el sistema seleccione, descarte o contrate.
- El tiempo disponible es el del calendario académico, sin personal de QA dedicado.

Dependencias y Riesgos

Dependencias: imágenes Docker publicadas (postgres, redis, cypress/included), dependencias de Composer y npm fijadas por sus archivos lock, GitHub Actions y el entorno virtual del servicio ML.

ID	Riesgo	Probabilidad	Impacto	Mitigación	Contingencia
R-01	El reset E2E borra datos de demostración	Baja	Alto	Entorno E2E aislado y rechazo del reset fuera de la base E2E (DEF-13)	Restaurar con migrate: fresh --seed en el entorno de demostración

ID	Riesgo	Probabilidad	Impacto	Mitigación	Contingencia
R-02	Pruebas inestables por fechas y zona horaria	Media	Medio	Fechas en APP_TIMEZONE (DEF-12) y sin reintentos	Clasificar como flaky y repetir la suite completa
R-03	Falla externa de la CI o de la red	Media	Medio	Misma suite ejecutable en local con Docker	Re-ejecutar la CI y registrar la ejecución local
R-04	Disponibilidad limitada del equipo	Media	Medio	Automatización y ejecución con un solo comando por herramienta	Priorizar criterios CA-01 a CA-06
R-05	Rendimiento y disponibilidad sin verificar	Alta	Medio	Se declaran NO VERIFICADOS (RNF-06 y RNF-07)	Recomendación de pruebas de carga como trabajo futuro
R-06	Premisa del AS-IS no validada por el Colegio	Alta	Medio	Se declara AS-IS PRELIMINAR	Validación institucional como trabajo futuro
R-07	Interpretar RF-29 como evaluación de personas	Baja	Alto	Pruebas que comprueban que la salida no menciona candidatos ni cambia el ranking	Desactivar el servicio: la aplicación funciona igual

Referencias

- Plantilla de Plan de Pruebas de Software del curso (docs/academico/00-fuentes-oficiales/plan-pruebas/).
- Referencia externa PMO Informática y ejemplo externo de plan de pruebas (no normativos).
- Especificación de requerimientos: Formatos 06 (RF) y 07 (RNF); casos de uso: Formato 08.
- Alcance: Formato 09. Arquitectura: Formato 11 (F11-R). Variables y operacionalización: F29C.
- Diseño y arquitectura tecnológica: docs/final-report/06-diseno-sistema.md y 07-arquitectura-tecnologica.md.
- Estrategia y automatización de pruebas de la v1.0: docs/final-report/11 a 13.
- Trazabilidad: docs/final-report/traceability-master.md; TDD: docs/tdd-evidence.md; defectos: docs/defects.md.
- Suite E2E: docs/testing/cypress-e2e.md. Entorno: docs/docker.md. QA de release v1.1: docs/v1.1/phase-25-final-qa.md.
- Norma ISO/IEC 25010, usada solo como marco de referencia de calidad (sin certificación).

Glosario

Término	Definición
CP	Caso de prueba (CP-001 en adelante), definido en la F29E
CU	Caso de uso del Formato 08 (CU-01 a CU-20)
RF / RNF	Requerimiento funcional (RF-01 a RF-27) y no funcional (RNF-01 a RNF-10)
TDD	Desarrollo guiado por pruebas: prueba que falla, implementación mínima y refactorización
E2E	Prueba de extremo a extremo en navegador (Cypress)
Regresión	Repetición de pruebas ya aprobadas para detectar fallos introducidos por cambios
Multitenencia	Aislamiento de los datos de cada organización (organization_id)
Flaky	Prueba que falla de forma intermitente sin cambios en el código
CI	Integración continua: ejecución automática de pruebas en cada push (GitHub Actions)
RACI	Responsable, Aprueba, Consultado, Informado
EXPERIMENTAL / PROPUESTO	Estado de RF-29: validado solo con datos sintéticos, fuera de la línea base